

EFFICIENCY, DECOMPOSITION, DATA EXAMPLES Solutions

COMPUTER SCIENCE MENTORS 61A

October 28 – October 31, 2025

1 Efficiency and Orders of Growth

1. Give a tight asymptotic runtime bound for the following functions in $\Theta(\cdot)$ notation, or “Infinite” if the program does not terminate.

(a) `def one(n):`
 `while n > 0:`
 `n = n // 2`

$\Theta(\log n)$

(b) `def two(n):`
 `for i in range(n):`
 `for j in range(i):`
 `print(str(i), str(j))`

$\Theta(n^2)$

(c) `def three(n):`
 `i = 1`
 `while i <= n:`
 `for j in range(i):`
 `print(j)`
 `i *= 2`

$\Theta(n)$

2. What do these runtimes simplify to?

$\Theta(8n^2)$	
$\Theta(10000)$	
$\Theta(100n^2 + 10n^3)$	
$\Theta(3n^2 * 2^n)$	
$\Theta(n^{30} + 3^n)$	
$\Theta(n \log n + \log n + n)$	

$\Theta(8n^2)$	$\Theta(n^2)$
$\Theta(10000)$	$\Theta(1)$
$\Theta(100n^2 + 10n^3)$	$\Theta(n^3)$
$\Theta(3n^2 * 2^n)$	$\Theta(n^2 * 2^n)$
$\Theta(n^{30} + 3^n)$	$\Theta(3^n)$
$\Theta(n \log n + \log n + n)$	$\Theta(n \log n)$

2 Review

1. Fill in the implementation of double_link.

```
def double_link(lnk):
    '''Using mutation, replaces the second in each pair of items
    with the first. The first of each pair stays as is.

    >>> double_link(Link(1, Link(2, Link(3, Link(4)))))
    Link(1, Link(1, Link(3, Link(3))))
    >>> double_link(Link('c', Link('s', Link(6, Link(1, Link('a'))))))
    Link('c', Link('c', Link(6, Link(6, Link('a')))))
    '''

    if lnk is Link.empty or lnk.rest is Link.empty:
        return lnk
    lnk.rest.first = lnk.first
    double_link(lnk.rest.rest)
    return lnk
```

2. Write a function that returns `True` if there exists a path from root to leaf that contains at least `n` instances of `elem` in a tree `t`.

*Hint: recall that the built-in function **any** takes in an iterable and returns `True` if any of the iterable's elements are truthy.*

```
def contains_n(elem, n, t):
    '''
    >>> t1 = Tree(1, [Tree(1, [Tree(2)])])
    >>> contains_n(1, 2, t1)
    True
    >>> contains_n(2, 2, t1)
    False
    >>> contains_n(2, 1, t1)
    True
    >>> t2 = Tree(1, [Tree(2), Tree(1, [Tree(1), Tree(2)])])
    >>> contains_n(1, 3, t2)
    True
    >>> contains_n(2, 2, t2) # Not on a path
    False
    '''
    if n == 0:
        return True

    elif _____:
        return _____

    elif _____:
        return _____

    else:
        return _____
```

```

if n == 0:
    return True
elif t.is_leaf():
    return n == 1 and t.label == elem
elif t.label == elem:
    return any([contains_n(elem, n - 1, b) for b in
                t.branches])
else:
    return any([contains_n(elem, n, b) for b in
                t.branches])

```

Base cases: The simplest case we have is when $n == 0$, or when we want at least 0 instances of `elem` in `t`. In this case, we always return `True`. The other simple case we consider is when the tree is only a leaf — there is nothing left to recurse on. In that case, we simply check to see that both $n == 1$ and that `t.label == elem`, meaning that we have one element left to satisfy, and the leaf label satisfies the final element we are looking for. If we have more elements to search for (ie. $n \geq 1$), then we will not satisfy that many elements at the leaf node; conversely, if we have fewer (ie. $n == 0$), then the case would already be covered by the first base case.

Recursive cases: If the current node isn't a leaf, then there's two different cases we should consider. Either the label of the current node is equal to `elem` or the label is not equal to `elem`. For the former, we would have to search for n more elems in each branch of `t` and return `True` if any of the branches contain n elems. For the latter, we would have $(n - 1)$ elements remaining, so we would search for $(n - 1)$ more elems in each branch of `t` and return `True` if any of the branches contain $(n - 1)$ elems. Since there is not room to do a for loop, we can use a list comprehension to recursively call the function on each branch. Thus, our two list comprehension statements would be `[contains_n(elem, n, b) for b in t.branches]` and `[contains_n(elem, n - 1, b) for b in t.branches]`. To determine if any of the branches contain either n elems or $(n - 1)$ elems, we can check if there's a `True` element in the respective lists.

3. Produce the environment diagram and output that result from executing the code below.

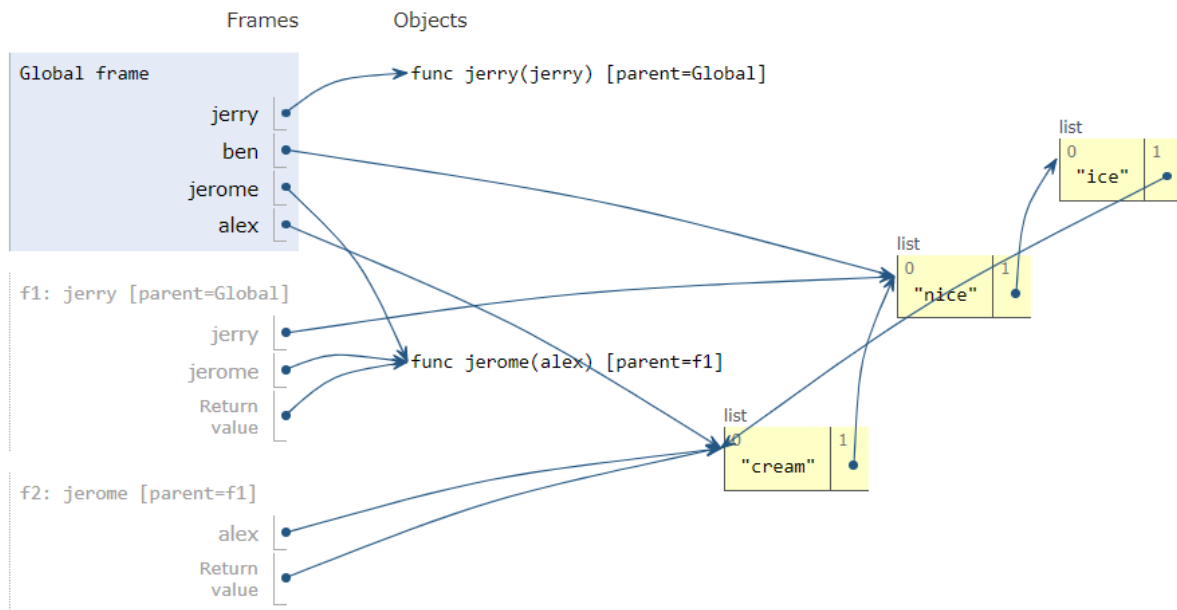
```
def jerry(jerry):  
    def jerome(alex):  
        alex.append(jerry[1:])  
        return alex  
    return jerome
```

```
ben = ['nice', ['ice']]  
jerome = jerry(ben)  
alex = jerome(['cream'])  
ben[1].append(alex)  
ben[1][1][1] = ben  
print(ben)
```

<https://goo.gl/uhSCLR>

Print output (drag lower right corner to resize)

```
['nice', ['ice', ['cream', [...]]]]
```



4. Write a generator that, given *m* (the amount of money you have), *pc* (the cost of one pear), and *ac* (the cost of one apple) yields all possible combinations of fruit that you can buy that uses up the most of your money. In other words, each combination of fruits should not result in enough money left over to buy another fruit. Combinations of the same number of each fruit in different orders is okay. It is also okay if each combination has an extra space at the end.

```
def fruitOptions(m, pc, ac):
    '''
    >>> print(list(fruitOptions(10, 2, 5)))
    ['pear pear pear pear pear ', 'pear pear apple ', 'pear apple pear ',
    'apple pear pear ', 'apple apple ']
    '''
    if _____:
        yield ' '
    if m >= pc:
        for _____:
            _____
    if m >= ac:
        for _____:
            _____

def fruitOptions(m, pc, ac):
    if m < pc and m < ac:
        yield ' '
    if m >= pc:
        for p in fruitOptions(m-pc, pc, ac):
            yield 'pear ' + p;
    if m >= ac:
        for a in fruitOptions(m-ac, pc, ac):
            yield 'apple ' + a;
```